Expert Systems With Applications 326 (2026) 132563

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Multi-dimensional scheduling parameter search for efficient multi-model inference on underground mine edge devices

Lei He [a], Yuelong Li [a,b,1], Weijun Cheng [a,c,*]

[a] School of Information Engineering, Minzu University of China, No. 27, South Zhongguancun Street, Beijing, 100081, China
[b] Yunnan Tin and Indium Laboratory, Kunming, 650000, China
[c] Institute of National Security, Minzu University of China, No. 27, South Zhongguancun Street, Beijing, 100081, China

ARTICLE INFO

Keywords:
Mine safety
Scheduling parameter search
Edge intelligence
Multi-model inference

ABSTRACT

With the advancement of intelligent coal mine construction, mine safety monitoring is evolving from cloud to edge, increasingly relying on multi-model systems integrating object detection, behavior recognition, and environmental prediction to enhance perception and safety warnings. However, executing multiple deep learning models concurrently on resource-constrained edge devices poses a significant challenge, often resulting in high latency and inefficient resource utilization. To address this challenge, we propose MineInfer, a lightweight multi-model inference framework specifically designed for edge devices. MineInfer leverages fine-grained concurrency scheduling, resource load balancing, and a standardized inference interface to enable collaborative inference of multiple monitoring models on a single device, thereby reducing deployment and maintenance overhead while improving real-time responsiveness. Furthermore, an automatic scheduling optimization strategy based on scheduling parameter search is introduced, which constructs a multi-dimensional parameter search space and combines random search with genetic algorithms to adaptively optimize scheduling schemes on edge devices. Experimental results on representative edge platforms demonstrate that MineInfer significantly outperforms traditional serial and concurrent inference strategies, achieving a 1.3-2.5× reduction in end-to-end inference latency while enhancing stability and resource utilization. These results confirm the effectiveness, adaptability, and generalizability of the proposed framework, indicating its potential for real-world multi-task intelligent monitoring applications in heterogeneous edge environments.

1. Introduction

Coal has long been recognized as a cornerstone of the energy system and continues to serve as one of the dominant sources of primary energy consumption worldwide. However, coal mining environments are typically characterized by confined working spaces, poor ventilation, and densely arranged equipment, which collectively create conditions prone to severe safety hazards, such as fires, gas explosions, equipment failures, and human operational errors (Ali & Pal, 2022). Therefore, the development of an efficient and intelligent safety monitoring system is imperative to enhance operational safety and ensure the stable operation of mining activities.

With the continuous advancement of intelligent mine construction, artificial intelligence (AI) technologies are being increasingly incorporated into underground safety monitoring systems (Shi et al., 2024). The deployment of deep learning models enables these systems to perform real-time analyses of collected images, video streams, and multi-source sensor data (Liu et al., 2025). Such capability facilitates comprehensive perception and timely risk alerts related to personnel behaviors, equipment operations, and environmental parameters. For instance, in coal conveyor belt systems, object detection and image classification models are employed to identify risks such as foreign objects embedded in coal and potential fire hazards (Zhang et al., 2024a). In regions prone to gas accumulation, time-series prediction models are utilized to dynamically characterize environmental data, thereby enabling early anomaly detection (You et al., 2021). In mining scenarios, owing to the diversity and complexity of task requirements, no single model can effectively address all monitoring objectives. In response, certain studies have explored unified intelligent paradigms to manage heterogeneous monitoring demands. For example, Sun et al. proposed an embodied safety assessment framework that integrates perception modules with hierarchical reasoning and domain knowledge, leveraging large-scale models to enable interactive inference, anomaly interpretation, and long-term risk analysis (Sun & Ji, 2025). While such Large Language Model-driven

* Corresponding author.
E-mail addresses: 24302167@muc.edu.cn (L. He), yuelong59@163.com (Y. Li), weijuncheng@muc.edu.cn (W. Cheng).
[1] Make the same contribution as the first author.

https://doi.org/10.1016/j.eswa.2026.132563
Received 9 December 2025; Received in revised form 28 March 2026; Accepted 20 April 2026
Available online 2 May 2026
0957-4174/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

(LLM-driven) systems enhance cognitive capabilities and decision interpretability, their substantial computational costs and dependence on centralized resources hinder their deployment on resource-constrained edge devices. Consequently, the development of a multi-model collaborative inference framework that integrates diverse functional models has emerged as a key research trend in intelligent mine monitoring. By integrating multiple models-such as object detection, image classification, and time-series prediction-intelligent monitoring and multi-dimensional decision-making can be achieved on a unified platform, significantly enhancing the safety and responsiveness of underground operations.

Currently, most intelligent coal mine monitoring systems adopt a deployment paradigm of edge data collection, cloud uploading, and centralized analysis, in which cloud servers are responsible for processing large volumes of underground data in a centralized manner. Although cloud platforms provide powerful computational capabilities, the remote transmission of massive underground data relies heavily on network bandwidth and stability, resulting in substantial communication overhead. Deploying intelligent monitoring models on edge devices located at data collection points can effectively reduce communication overhead and enable real-time anomaly detection and response (Li et al., 2024).

However, edge devices possess constrained computing resources and are incapable of efficiently supporting the concurrent execution of multiple compute-intensive models. In this context, the realization of efficient collaborative inference among diverse AI models on a single edge device has emerged as a central challenge for intelligent mine monitoring systems. At this stage, certain studies have demonstrated multi-model parallel inference on high-end platforms equipped with flexible scheduling capabilities, including the NVIDIA Xavier NX and FPGA. For example, Zhang et al. utilized CUDA streams on the Xavier NX to implement operator-level, fine-grained concurrent scheduling (Zhang et al., 2023), while Kim et al. developed a hardware-software co-scheduling engine on the FPGA (Kim et al., 2023). These platforms are costly, power-intensive, characterized by high deployment expenses, and challenging to maintain in mining environments, thereby restricting their large-scale applicability. In contrast, the LubanCat-4 embedded development board, built upon the Rockchip RK3588 platform, represents an attractive solution for edge intelligent monitoring in mines, owing to its low cost (approximately one-seventh of that of the Xavier NX), 6-TOPS INT8 computing capacity, and robust industrial adaptability. However, the proprietary RKNN (Airockchip, 2025) inference framework employed on this platform does not natively support multi-model concurrency and lacks flexible scheduling interfaces, thereby rendering direct migration nearly infeasible.

In this article, to minimize the construction and maintenance costs of mine monitoring systems, MineInfer is proposed as an edge-side multi-model inference framework built upon the RKNN platform. It enables the integrated deployment of multiple AI models on a single edge device, thereby fulfilling the multi-task intelligent inference requirements of underground mine monitoring. The MineInfer framework adopts a three-stage progressive architecture. At the first stage, our previous MineNAS lightweight model (Li et al., 2024) is integrated to decompose complex networks into statically schedulable subgraphs, which are organized into a scheduling computation graph to achieve fine-grained concurrency through a thread-pool mechanism. At the second stage, CPU task affinity binding, dynamic thread pools, and zero-copy memory mechanisms are introduced to achieve resource load balancing and low-overhead inference. At the final stage, a unified abstract inference interface is designed to encapsulate model loading, data preprocessing, and result analysis, thereby improving development and deployment efficiency. To further enhance the performance of the framework, an automatic scheduling optimization strategy based on scheduling parameter search is developed. This strategy defines a scheduling parameter space that includes inference-stage concurrency, model prioritization, and NPU core allocation. A latency cost model, driven by performance profiling, is integrated with search algorithms to realize optimal scheduling on edge devices.

This article is centered on intelligent multi-model scheduling in resource-constrained edge environments. An edge-oriented multi-model inference framework, termed MineInfer, together with a scheduling optimization strategy, is proposed. The main contributions of this article are as follows:

- To enable intelligent monitoring of diverse underground mining tasks, the MineInfer framework is proposed as an edge-side multi-model inference system. Multiple monitoring models are jointly deployed on a single edge device, achieving integrated multi-model inference with low resource overhead and high task adaptability.
- An automatic scheduling optimization strategy based on scheduling parameter search is developed to improve inference performance on edge devices. A scheduling parameter space is constructed, where a latency-cost model is integrated with search algorithms to achieve optimized edge-side scheduling and enhanced overall inference efficiency.
- Experimental results demonstrate that the proposed MineInfer framework, with automatic scheduling optimization, outperforms three existing multi-model inference schemes—serial inference (Airockchip, 2025), simple multi-concurrent inference (Huang et al., 2020), and modular concurrency (Nndeploy, 2025)—in terms of inference speed and resource utilization. Compared with the native serial inference mechanism, MineInfer reduces inference latency by 1.3-2.5× across various task combinations.

The remainder of this article is structured as follows. In Section 2, the related literature on intelligent mine monitoring and multi-model inference optimization for edge devices is reviewed. Section 3 presents the design and implementation of the proposed MineInfer framework; we emphasize its modular architecture and task scheduling mechanisms. An automated scheduling strategy based on scheduling-parameter search is developed in Section 4 to further enhance inference efficiency. Section 5 describes the experimental setup and presents the evaluation results, where we demonstrate that the framework effectively reduces latency and improves resource utilization. Finally, Section 6 concludes the paper and outlines possible directions for future research.

2. Related work

This section reviews the current applications and research progress of artificial intelligence (AI) models in intelligent mine monitoring systems. The discussion then extends to multi-model inference techniques and their optimization strategies, with an emphasis on deployment within resource-constrained edge environments.

2.1. Intelligent mine monitoring systems

The clarity of underground mine images is often degraded by poor illumination and dense dust particles, while traditional manual monitoring is prone to subjective misjudgment. To address these challenges, artificial intelligence (AI) techniques, including object detection, image classification, and time-series prediction, have been widely applied to intelligent mine safety monitoring for perceptual decision support.

Most existing studies focus on task-level optimization of visual perception models. Wu et al. proposed a lightweight small-object detection framework for coal mine IoVT systems based on multi-layer collaborative federated intelligence, which improved detection accuracy and reduced communication latency (Wu et al., 2024). Fan et al. developed the CM-YOLOv8 model for real-time monitoring of hydraulic support plate conditions in fully mechanized mining faces (Fan et al., 2024), while Wang et al. enhanced SSD with depthwise separable convolutions, GIoU loss, and optimized feature extraction for foreign-object detection on conveyor belts (Wang et al., 2023). For image classification, Luan et

al. combined CNNs with the Swin Transformer (Liu et al., 2021) for coal gangue identification (Luan et al., 2023), Kou et al. introduced an ESCBAM-based lightweight classification model for foreign-object recognition (Kou et al., 2023), and Liu et al. applied CNNs to flotation froth image analysis for ash content prediction (Liu et al., 2024). Although these methods achieve high accuracy in specific scenarios, they are predominantly model-centric and rely on manually designed architectures, making them difficult to scale across multiple concurrent perception tasks under shared hardware constraints.

Beyond static visual perception, time-series prediction and multi-source sensing have been explored for dynamic safety monitoring. Chang et al. employed a Transformer-based spatiotemporal model for gas concentration forecasting to enhance early warning capability (Chang et al., 2024). Duan et al. proposed a stacking ensemble of Conv1D-LSTM models for motor insulation life prediction under varying working conditions (Duan & Xu, 2024). Lalitha et al. designed an intelligent helmet integrating physiological and environmental sensing with CNN-based gesture recognition (Lalitha et al., 2023). Xu et al. combined InSAR with time-series classification using dynamic time warping for subsidence area recognition (Xu et al., 2024), while Cheng et al. integrated particle swarm optimization with neural architecture search to construct adaptive image recognition networks for complex mining environments (Cheng et al., 2025). These studies demonstrate a clear trend toward intelligent and automated perception; however, they still treat each task as an independent decision unit and rarely consider coordinated execution on resource-constrained edge platforms.

In summary, existing works generally emphasize single-task performance improvement while overlooking the system-level problem of jointly deploying multiple heterogeneous AI models under strict latency and resource constraints. In real underground mining environments, simultaneous execution of detection, classification, and time-series prediction is required for continuous safety decision-making. To address this critical gap, this article proposes a unified scheduling-search-driven framework for multi-model inference and deployment, enabling efficient edge-side scheduling for real-time intelligent mine monitoring.

2.2. Optimization of multi-model inference at the edge

Numerous studies have optimized multi-model inference scheduling strategies from the perspectives of model partitioning, task parallelism, and dynamic scheduling, often in combination with dynamic resource allocation mechanisms to achieve efficient task scheduling. Zhang et al. reformulated the multi-model inference latency minimization problem as a fine-grained operator scheduling task and employed operator fusion, subgraph reuse, inter- and intra-operator parallelism, together with reinforcement learning, to optimize execution order and concurrency (Zhang et al., 2023). Kim et al. introduced cross-layer optimization across hardware, runtime, and scheduling, dynamically adjusting memory access rates, runtime resource allocation, and task-level parallelism to reduce memory contention and inference latency (Kim et al., 2023). Huang et al. proposed the MPInfer framework, which supports pipeline and GPU parallelism combined with pageable and pinned memory strategies to improve data transfer efficiency (Huang et al., 2020). Jeong et al. partitioned multiple DNN models into scheduling graphs and adopted a pluggable runtime scheduling strategy to dynamically determine execution order and parallel schemes (Jeong et al., 2022). Yu et al. proposed a machine learning-based GPU multi-tenant framework that builds a unified scheduling representation and applies automatic search to regulate concurrency and interleaved execution for balanced resource utilization (Yu et al., 2021). Zhang et al. presented BCEdge, an adaptive reinforcement-learning-based scheduling framework that coordinates batch size, concurrency, and memory configuration (Zhang et al., 2024b). Zhao et al. and Zou et al. further proposed elastic kernel mechanisms by decomposing GPU kernels into fine-grained execution units, enabling more flexible resource control (Zhao et al., 2023; Zou et al., 2023). Xue et al. exploited idle cycles caused by unbalanced utilization of matrix multiplication and vector units during inference, and proposed a learning-based task clustering mechanism for intelligent task matching and efficient resource sharing (Xue et al., 2023).

Collectively, these studies demonstrate that fine-grained operator scheduling, elastic kernel design, and learning-based concurrency control are highly effective for improving multi-model inference performance on open and programmable GPU platforms. However, their effectiveness critically depends on full access to kernel execution, memory hierarchy, and low-level hardware scheduling mechanisms.

In parallel, edge-oriented lightweight model design and NAS-based efficiency optimization have been explored to reduce single-model inference overhead. Hanif et al. proposed a lightweight detection model for underground mining through pruning and depthwise separable convolutions, achieving over 40% parameter reduction while maintaining over 89% accuracy under sub-50 ms latency (Hanif et al., 2024). Zhang et al. proposed OnceNAS, an edge-oriented NAS framework based on OFA and progressive shrinking that enables flexible adaptation of depth, width, kernel size, and resolution while reducing computation and energy consumption (Zhang et al., 2024c). However, these studies focus on optimizing individual models and do not address the scheduling conflicts and resource contention induced by concurrent execution of multiple heterogeneous models.

From a hardware perspective, existing multi-model scheduling methods are mainly designed for two categories of platforms: open programmable GPUs and reconfigurable FPGAs. GPUs support aggressive operator reordering and memory reuse through software schedulers, while FPGAs enable hardware-software co-optimization via reconfigurable logic. Although these approaches have significantly advanced multi-model scheduling on high-performance hardware, they are largely unsuitable for closed-source edge NPUs, which are constrained by restricted hardware access, limited operator-level control, fixed execution pipelines, and tight power and memory budgets. In practice, most commercial edge NPUs adopt closed-source designs, making dynamic operator reconfiguration, elastic kernel scheduling, and fine-grained hardware-aware optimization difficult or even infeasible. Accordingly, this article targets edge devices equipped with closed-source NPUs and proposes a multi-model scheduling framework that supports fine-grained scheduling and load balancing to improve overall inference efficiency.

3. MineInfer: A multi-model inference scheduling framework

This article aims to achieve efficient multi-model inference on edge devices. A common serial execution scheme, as illustrated in Fig. 1(a), executes inference tasks for multiple models in sequence. Although the serial execution scheme is easy to implement, it cannot fully exploit the parallel computing capabilities of multi-core CPUs and NPUs. This limitation leads to idle computational resources and high inference latency, hindering real-time responsiveness. To improve efficiency, a basic multi-threaded concurrent scheme allows multiple threads to execute inference tasks of different models in parallel, reducing overall latency. However, as shown in Fig. 1(b), this approach lacks fine-grained scheduling and cross-stage coordination, leading to thread contention and limited performance improvement. A further optimized scheme, illustrated in Fig. 1(c), divides the inference process of a single model into three stages—preprocessing, inference, and postprocessing—and employs a pipeline mechanism to achieve inter-stage concurrency. Although resource utilization improves, uneven stage workloads cause bottlenecks and idle waiting, thereby limiting overall performance. To address these challenges, this article introduces the MineInfer inference scheduling framework, as shown in Fig. 1(d). The framework uses an interleaved execution strategy for DNN operations, enabling highly concurrent scheduling across models and stages. By integrating fine-grained task decomposition, dynamic thread pooling, and resource load balancing, MineInfer maximizes edge device utilization, mitigates memory contention, and minimizes idle compute cycles.

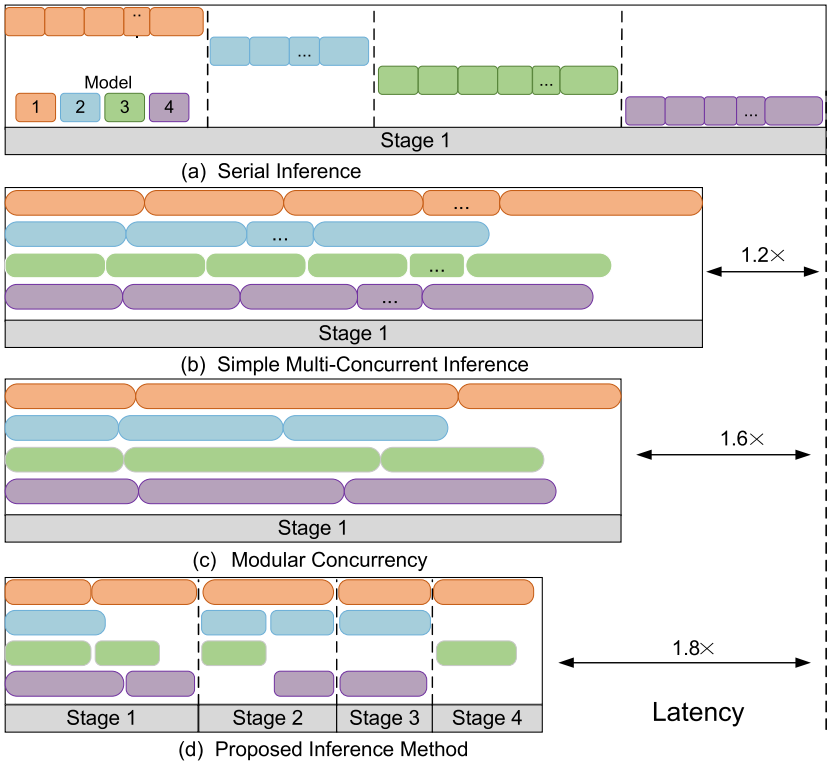


Fig. 1. Multi-model inference modes diagram.

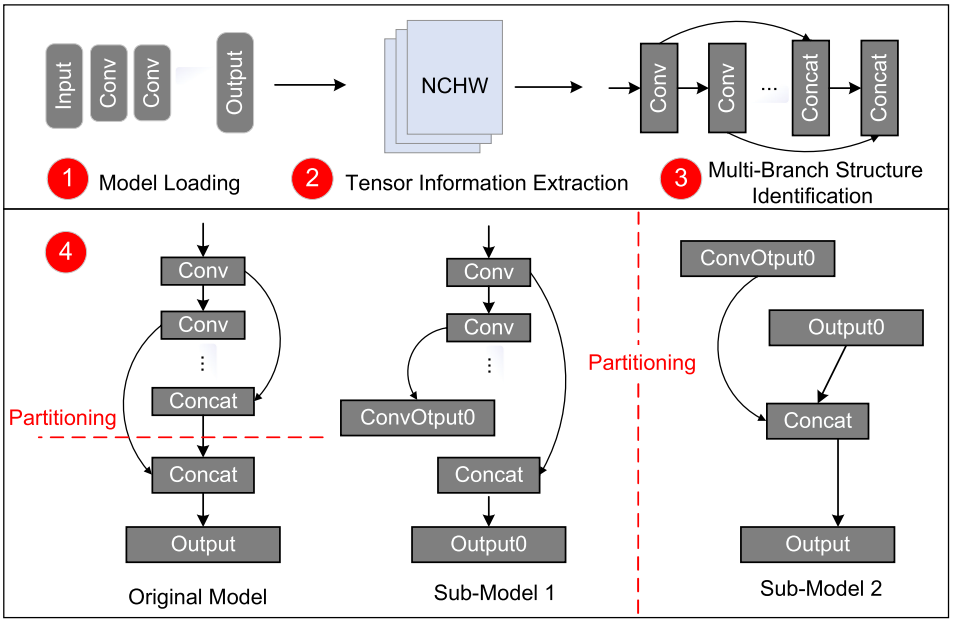


Fig. 2. Workflow of the model segmentation module.

3.1. Functional design and implementation

The MineInfer framework is built upon the RKNN inference interface and integrates flexible scheduling strategies to enable efficient execution of complex multi-model tasks. The computation graph is partitioned for sequential scheduling under dependency constraints, while fine-grained coordination is achieved through stage concurrency and priority control. Zero-copy technology reduces data transfer overhead, and a dynamic thread pool combined with big-little core mapping ensures adaptive and efficient resource utilization. Moreover, a set of abstracted interfaces-covering model management, task scheduling, data preprocessing, inference execution, and result output-forms a unified operational layer that hides the heterogeneity of different models.

3.1.1. Model partitioning module design

The model inference process is typically divided into three stages: preprocessing, inference, and postprocessing. For instance, nndeploy (Nndeploy, 2025) adopts this three-stage partitioning to implement

multi-model inference scheduling. However, this coarse-grained approach overlooks variations in resource demands across heterogeneous computational segments, thereby preventing optimal resource allocation. To address this limitation, a dedicated model-segmentation module is incorporated into the framework. By dividing the model into smaller segments, the scheduling granularity is refined from the whole-model to the sub-model level, thus enabling more fine-grained scheduling. The workflow of this module is illustrated schematically in Fig. 2. The Open Neural Network Exchange (ONNX) model is loaded from a specified path, and the shape information of input and output tensors is extracted to ensure interface consistency among sub-models after segmentation. During computation-graph parsing, all graph nodes are traversed to identify and record operators related to multi-branch structures. In the segmentation phase, segmentation nodes are detected and additional input nodes introduced, with information mapping established between consecutive nodes. Following segmentation, the resulting sub-models are exported in ONNX format, each containing computational operators, I/O specifications, and initialization parameters to facilitate subsequent inference management and deployment. Consequently, the original model is divided into multiple statically schedulable sub-model units, compensating for the lack of a computation-graph partitioning interface in RKNN and enabling operator-level partitioning and scheduling.

Based on the conceptual model partitioning workflow illustrated in Fig. 2, the original model set M must be formally transformed into a structured set of executable tasks to support fine-grained scheduling. This transformation involves a systematic traversal of the ONNX computation graphs G_i to extract tensor attributes, identify complex multi-branch structures, and determine optimal partition boundaries. The precise implementation of this process-covering graph parsing, subgraph exportation, and task registration-is formalized in Algorithm 1. By generating the candidate task set T_{cand} and the topological dependency set D, this algorithm establishes the structural foundation required for the subsequent multi-dimensional scheduling parameter search.

Algorithm 1 Model partitioning and candidate task set generation.

Require: Model set $M = \{M_1, M_2, \ldots, M_N\}$
Ensure: Candidate task set T_{cand} and dependency set D
Initialize $T_{\text{cand}} = \emptyset$, $D = \emptyset$
for each model M_i in M **do**
  Load ONNX computation graph $G_i = (V_i, E_i)$
  Extract I/O tensor attributes and shapes for G_i
  Traverse G_i to identify multi-branch operators and partition points
  Partition G_i into ordered subgraphs $\{g_{i1}, g_{i2}, \ldots, g_{iK_i}\}$
  for each subgraph g_{ij} **do**
    Export g_{ij} as a sub-model s_{ij}
    Create a scheduling task t_{ij} and register its execution function
    Bind t_{ij} with I/O specifications and unique TaskID
    Add t_{ij} to T_{cand}
  Construct topology D_i based on G_i and add to D
Return T_{cand}, D

The computational complexity of the model partitioning and candidate task set generation (Algorithm 1) is linearly proportional to the structural scale of the input DNN computation graphs. Let $G_i = (V_i, E_i)$ denote the computation graph of model M_i, where $|V_i|$ and $|E_i|$ represent the number of operator nodes and dependency edges, respectively. Since the partitioning process—including graph parsing, multi-branch identification, and task registration—requires only a constant number of traversals over the graph components, the total time complexity for a task group containing N models is $O\left(\sum_{i=1}^{N}(|V_i| + |E_i|)\right)$. Correspondingly, the space complexity is $O\left(\sum_{i=1}^{N}(|V_i| + |E_i|)\right)$ to accommodate the storage of sub-model metadata and the task adjacency list. As a one-time pre-processing step, this stage maintains high efficiency and ensures that the subsequent scheduling search in Algorithm 2 operates on a well-defined set of discrete task units.

3.1.2. Dependency graph module design

The dependency graph module is designed to manage task dependencies and to generate the correct execution order accordingly. In this module, an adjacency list is employed to store successor relationships, thereby ensuring proper handling of task dependencies during execution. An in-degree table is maintained to record the number of tasks on which each task depends. The in-degree value is utilized to determine whether a task is eligible for direct execution. The task mapping table associates each task ID with its corresponding execution function, thereby enabling the scheduler to invoke tasks efficiently during execution. The task dependency table is constructed to record predecessor tasks in reverse order, thereby facilitating efficient dependency management. The execution result mapping table is designed to store the outputs of executed tasks, thereby enabling dependent tasks to access the necessary data. The design of the dependency graph module is shown in Fig. 3.

Beyond the structural design, the dependency-graph module also incorporates a dynamic management mechanism to ensure consistent and deadlock-free task execution. During system operation, task dependencies within the dependency-graph module are managed via a three-tier mechanism that encompasses dynamic readiness determination, circular-dependency prevention, and result-oriented delivery. The dependency status of each task is dynamically updated through real-time maintenance of an in-degree table. Once the in-degree of a task decreases from a positive value to zero, it is immediately marked as ready and enqueued for scheduling, ensuring that execution does not commence until all prerequisite tasks have completed. To prevent deadlocks, a topological pre-check is executed during initialization to identify cyclic dependencies recorded in the adjacency list. When closed-loop relations (e.g., $A \rightarrow B \rightarrow C \rightarrow A$) are detected, the system automatically reports a dependency error and pinpoints the circular nodes, allowing developers to refine task logic prior to execution. In addition, execution results are delivered through a result-mapping table instead of global broadcasting, ensuring that the output of each task is accessible solely to its direct dependents. This design reduces memory usage and avoids concurrent data-access conflicts, which is particularly advantageous for edge devices with limited memory bandwidth. The dependency-handling mechanism operates synergistically with the stage-concurrency and priority-scheduling strategies of the scheduler. Ready tasks are dispatched to designated thread pools according to their stage and priority, while an NPU-core affinity mechanism alleviates resource contention, thereby improving the stability and efficiency of multi-model collaborative inference.

3.1.3. Design of the model scheduler module

The model scheduler is designed to perform fine-grained task scheduling based on task dependencies and the results of topological sorting. To maximize resource utilization and improve overall system performance, three scheduling strategies are incorporated: task-stage concurrency, priority-based scheduling, and NPU-core binding. The concepts of task-stage concurrency and priority scheduling are illustrated in Fig. 4.

1. Task stage concurrency

In multi-model inference tasks, each model is typically partitioned into several sub-models with substantially varying inference times and resource utilizations. If a particular type of sub-model exhibits prolonged inference time while failing to fully utilize resources, idle resources can be reallocated by the system to schedule other sub-model types. For illustration, consider the task set $T = \{A_1, A_2, A_3, A_4, B_1, B_2, B_3, C_1, C_2, C_3, D_1, D_2\}$. Suppose concurrency is not phase-controlled and all tasks are submitted simultaneously; this may result in resource contention and task blocking (i.e., waiting among dependent tasks), thereby degrading overall performance. To address

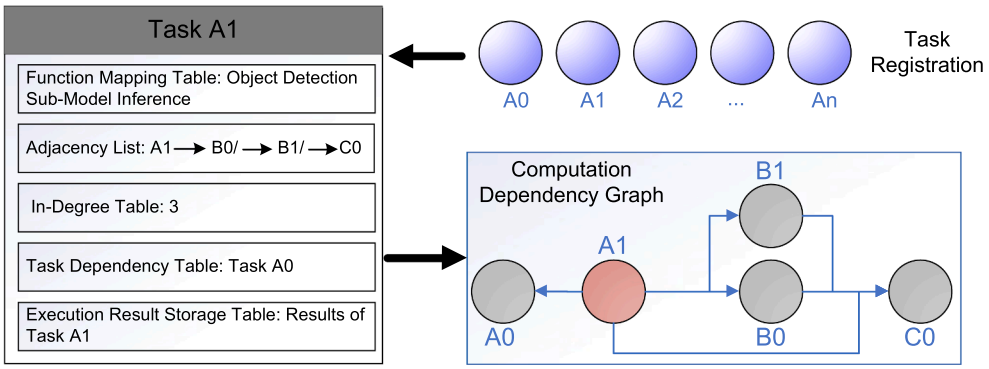


Fig. 3. Implementation of the dependency graph module.

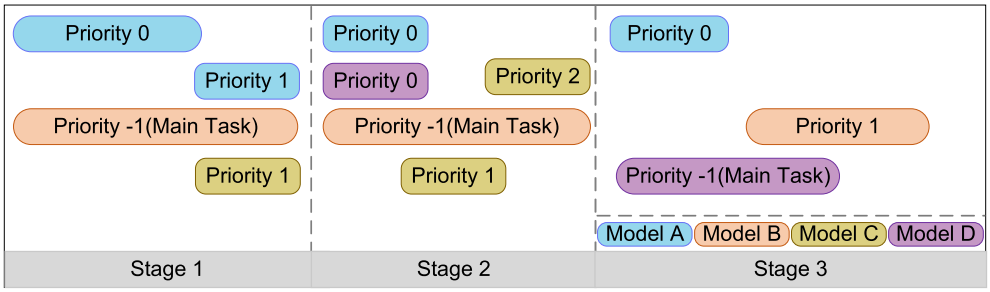


Fig. 4. Stage-level concurrency and priority scheduling of models.

this issue, the task set is partitioned into multiple stages by the scheduling module. Within each stage, tasks are allowed to execute concurrently; however, progression to the next stage requires completion of all tasks in the preceding stage. For example, the task set can be partitioned into three stages, as presented in Eq. (1).

$$\text{Stage0}\{A_1, A_2, B_1, C_1\}, \text{Stage1}\{A_3, B_2, C_2, C_3, D_1\}, \text{Stage2}\{A_4, B_3, D_2\} \tag{1}$$

The task execution sequence is defined as Eq. (2).

$$\text{Schedule}: [\text{Stage0}, \text{Stage1}, \text{Stage2}] \tag{2}$$

Task-stage concurrency is inherently constrained by task dependencies. Therefore, the concurrency level of a subsequent task stage must not exceed that of its preceding stage. For example, the concurrency stage of task A_2 must align with that of task A_1, or be scheduled after the completion of A_1.

2. Priority scheduling

Based on the task-stage concurrency strategy, a priority scheduling scheme is introduced in this article to enable more fine-grained task scheduling. In each concurrency stage, all tasks are executed in accordance with their predefined priorities. For example, in Stage0, suppose task B_1 runs throughout the stage while consuming substantial resources. After submitting task A_1, the system does not have sufficient resources to execute task C_1. By applying the priority strategy, tasks C_1 and A_2 are assigned lower priorities than tasks B_1 and A_1. As a result, C_1 and A_2 must wait for the completion of task A_1 before running concurrently with the still-executing task B_1, thereby achieving a more balanced resource allocation. However, if the system strictly adheres to the priority order and the priority of task B_1 exceeds that of tasks C_1 and A_2, these three tasks cannot be executed concurrently.

To address this issue, the concept of mainline tasks and auxiliary tasks is proposed in this article. In each concurrency stage, at least one task is designated as a mainline task, whereas the remaining tasks are classified as auxiliary tasks. Auxiliary tasks must strictly follow the priority order, while mainline tasks are permitted to run concurrently with auxiliary tasks of different priorities. For example, in Stage0, task B_1 is designated as the mainline task, whereas the others are assigned as auxiliary tasks, thereby enabling concurrent execution of tasks C_1, A_2, and B_1. Additionally, within the same concurrency stage, the priority settings of tasks in the same category must conform to their inherent dependency relationships. Moreover, when tasks share equal priority, not all tasks within the same category may be designated as mainline tasks.

3.NPU core binding scheduling

Under a multi-core NPU architecture, tasks can be allocated to specific cores in a balanced manner, thereby preventing overload on certain cores while others remain underutilized. The framework flexibly schedules hardware resources according to the computational intensity and parallelism characteristics of each task. For computation-intensive tasks, multiple cores can be assigned to operate collaboratively, thereby accelerating execution. For lightweight tasks, a single core can be designated for inference to minimize inter-core data transfer, which would otherwise degrade inference performance. For example, the LubanCat-4 development board employed in this article supports five core configurations: (1) single-core mode with Core 1; (2) single-core mode with Core 2; (3) single-core mode with Core 3; (4) dual-core mode with Cores 1 and 2; and (5) full-load mode with all three cores activated. These diverse configurations enable flexible adaptation to the computational demands of heterogeneous tasks, thereby improving the flexibility and scalability of the system.

3.1.4. Thread pool module design

The thread pool module serves as the core execution unit of the MineInfer framework, responsible for managing and executing concurrent tasks. It employs a thread-safe task queue management system, consisting of a global queue and a local queue for each thread. The local queue stores tasks assigned to its corresponding thread, whereas the global queue maintains tasks not associated with any specific thread. A work-stealing mechanism is incorporated within the thread pool to enhance load balancing: when the local queue of a thread becomes empty, it first attempts to retrieve tasks from the global queue; if the global queue

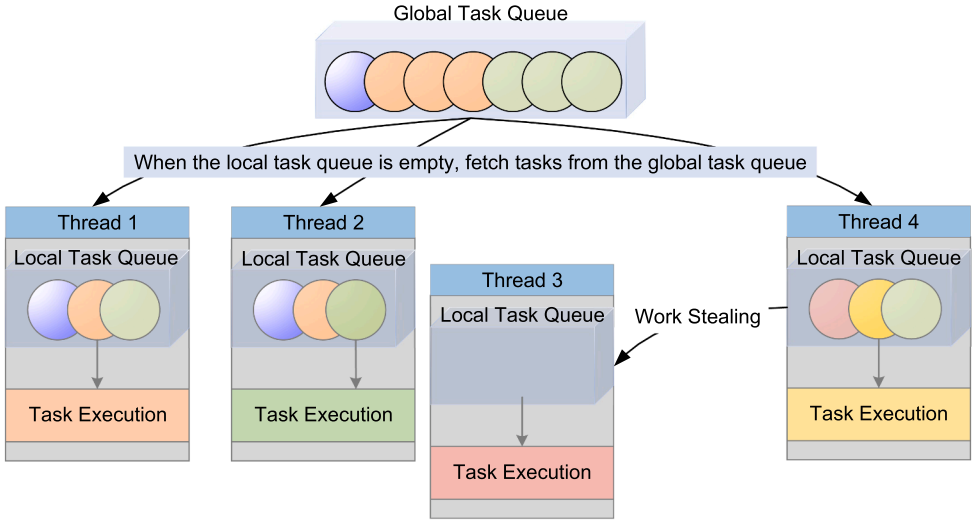


Fig. 5. Work-stealing mechanism in the thread pool.

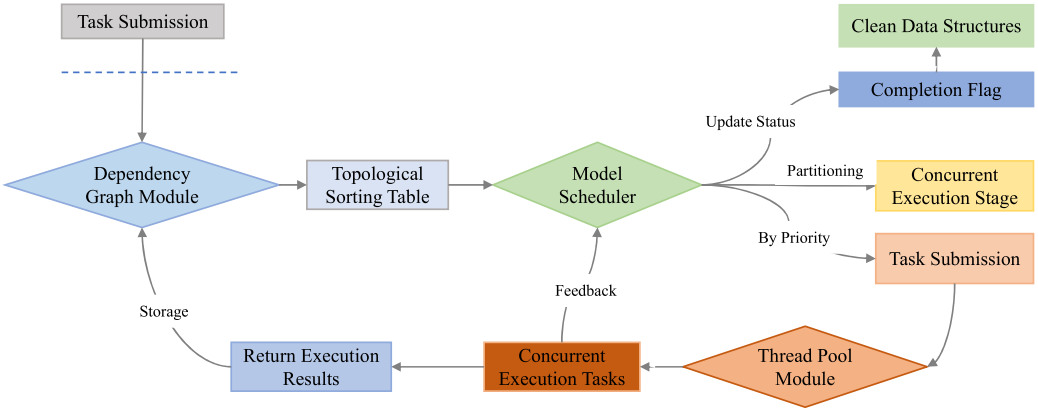


Fig. 6. Flowchart of the module cooperation mechanism.

is also empty, tasks are stolen from the local queues of other threads. This mechanism ensures balanced task allocation, preventing threads from remaining idle while avoiding overload in others. During execution, task functions are directly invoked, and results are recorded in the dependency-graph module for subsequent retrieval. Overall, this design improves resource utilization and enables efficient, reliable task execution through inter-thread cooperation. The work-stealing mechanism is illustrated in Fig. 5.

3.1.5. Module collaboration mechanism

The MineInfer framework facilitates efficient collaboration during task execution through three core modules: the Dependency Graph, Model Scheduler, and Thread Pool. The Dependency Graph Module manages task dependencies, the Model Scheduler orchestrates scheduling according to predefined priorities, and the Thread Pool Module executes inference tasks concurrently based on the scheduling strategy. These modules operate cooperatively to establish a cohesive and efficient task execution chain, as shown in Fig. 6.

Initially, tasks and their dependencies are dynamically defined by the user through the Dependency Graph Module. The module then constructs a topological ordering of tasks and forwards it to the Model Scheduler, which partitions the tasks into distinct concurrent execution stages and submits them to the Thread Pool according to their assigned priorities. During execution, the Thread Pool Module retrieves tasks from its queue, executes them, and records the results in the Dependency Graph Module for subsequent dependent tasks. Finally, the Model Scheduler updates the completion status of each task to ensure that the execution conditions of downstream dependencies are satisfied. Once all tasks are completed, temporary data structures in both the Dependency Graph and Scheduler Modules are cleared, thereby preparing the framework for the next round of task submissions.

3.2. Dynamic optimization of task and resource allocation

3.2.1. Zero-copy memory optimization strategy

To mitigate the performance bottleneck caused by redundant memory-copy operations during model inference, an input-output zero-copy mechanism based on the RKNN runtime was adopted. Specifically, pre-allocated tensor buffers provided by the RKNN NPU runtime were utilized to avoid unnecessary data movement between the CPU and NPU.

Before model execution, tensor attributes were queried using the `rknn_query` API with the `RKNN_QUERY_INPUT_ATTR` and `RKNN_QUERY_OUTPUT_ATTR` flags to obtain tensor dimensions, data layout (e.g., NCHW or NHWC), data types, and memory sizes. Based on the queried attributes, input and output memory buffers were explicitly allocated through the `rknn_create_mem` interface. These buffers were managed by the RKNN runtime, which maintained the corresponding virtual and physical address mappings required for NPU access.

After memory allocation, the allocated buffers were bound to the model using the `rknn_set_io_mem` API, enabling direct association between the model tensors and the pre-allocated memory regions. During

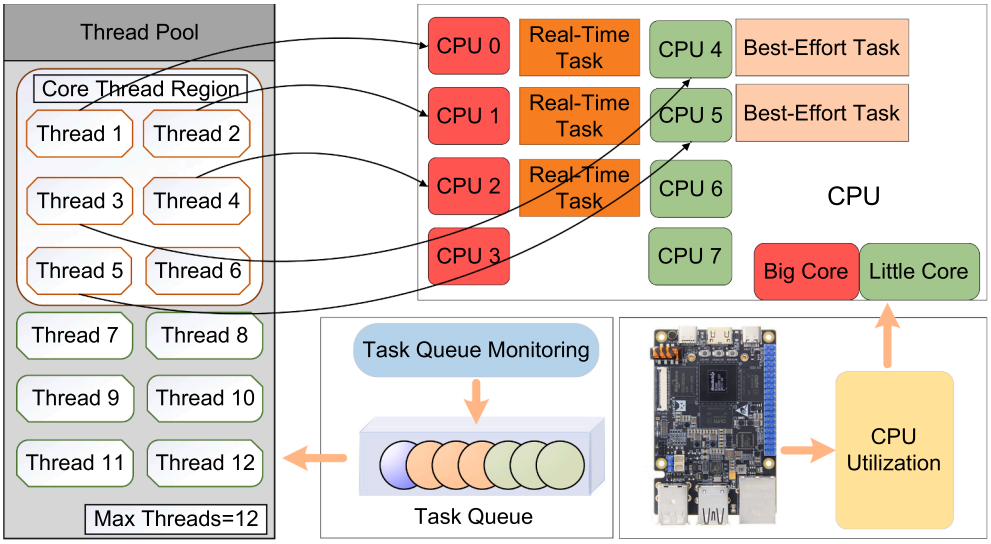


Fig. 7. Dynamic thread management and CPU task binding.

preprocessing, operations such as image resizing and format conversion were performed on the CPU, and the processed data were written directly into the bound input buffer without intermediate copying. During inference, the NPU read input data directly from the bound memory and wrote inference results into the pre-allocated output buffer. Finally, post-processing steps, including non-maximum suppression and result parsing, were executed directly on the output buffer, further reducing redundant memory-copy overhead.

3.2.2. *CPU task binding strategy*

The LubanCat-4 development board features an ARM big.LITTLE heterogeneous CPU architecture, consisting of four high-performance Cortex-A76 “big” cores and four high-efficiency Cortex-A55 “little” cores. To fully leverage the heterogeneous CPU architecture, the framework dynamically binds task threads to appropriate CPU cores according to their real-time requirements and the cores’ load conditions. Real-time tasks are scheduled on high-performance big cores to ensure low-latency processing, while best-effort tasks are assigned to the energy-efficient little cores to reduce power consumption and avoid contention for the big-core resources. CPU utilization, task-queue status, and latency metrics were periodically collected to construct a real-time load-estimation model. The thread-management module adjusts the thread-pool size and allocates tasks adaptively according to a core-load-aware scheduling strategy, thereby ensuring timely execution of critical workloads, as shown in Fig. 7.

3.3. *Inference interface abstraction design*

By applying an interface-based and modular design to the MineInfer inference pipeline, standardized inference logic can be reused across diverse task scenarios, thereby significantly simplifying the development process. The inference template construction process is shown in Fig. 8. The model management interface handles sub-model loading, initialization, and resource allocation; it also constructs runtime contexts, records I/O specifications and status, and enables unified management and adaptation of multiple models. The task management interface abstracts each inference task into an independent unit, where a globally unique ID is assigned via registration, and corresponding execution logic (e.g., inference, preprocessing, or post-processing) is bound. Task I/O formats, dependencies, and data flow are specified accordingly. The inference engine interface, serving as the core scheduler, constructs the computational graph based on dependencies. Tasks are executed concurrently via a thread pool, intermediate results are managed, and orderly and efficient inference execution is ensured. The data input and preprocessing interface collects raw data, standardizes it, and supports multiple formats (images, videos, sensor data) and common preprocessing operations such as letterboxing, normalization, and temporal segmentation. The model inference interface encapsulates the forward computation process, comprising an initial and a regular sub-model interface. The initial interface receives standardized input data, while the regular interface takes the outputs from preceding models, enabling adaptation to various task types such as detection, classification, and temporal prediction. Finally, the result output interface performs post-processing and generates standardized outputs, including non-maximum suppression and visualized rendering in detection, confidence-based ranking in classification, and continuous prediction visualization in temporal tasks.

In summary, a scalable and efficiently schedulable multi-model execution framework has been developed by MineInfer. This framework is structured around three core components—model management, task scheduling, and inference execution—and additionally integrates three functional interfaces: data preprocessing, inference execution, and result output. The framework facilitates reliable and efficient inference optimization on edge devices.

4. Automated optimization of multi-model inference scheduling based on scheduling parameter search

Achieving high inference efficiency in multi-model edge intelligence systems requires careful coordination of task execution order and hardware resource allocation. In the MineInfer framework, the overall performance of multi-model inference is jointly determined by several runtime scheduling factors, including inference stage partitioning, task priority assignment, and NPU core allocation. These factors interact in complex and non-linear ways, making manual tuning inefficient and often suboptimal. To address this challenge, this section formulates multi-model inference optimization as a scheduling optimization problem, in which scheduling parameters are automatically explored to minimize end-to-end inference latency. By constructing a structured scheduling parameter space and integrating a cost model with search algorithms, scheduling schemes are iteratively explored and optimized, thereby improving the overall performance and task adaptability of the MineInfer framework. The specific optimization objective is to minimize the overall latency of multi-model inference, as expressed in Eq. (3).

$$\mathcal{E}^* = \arg\min_{\mathcal{E}\in P_s} \mathcal{L}(\mathcal{E}) \tag{3}$$

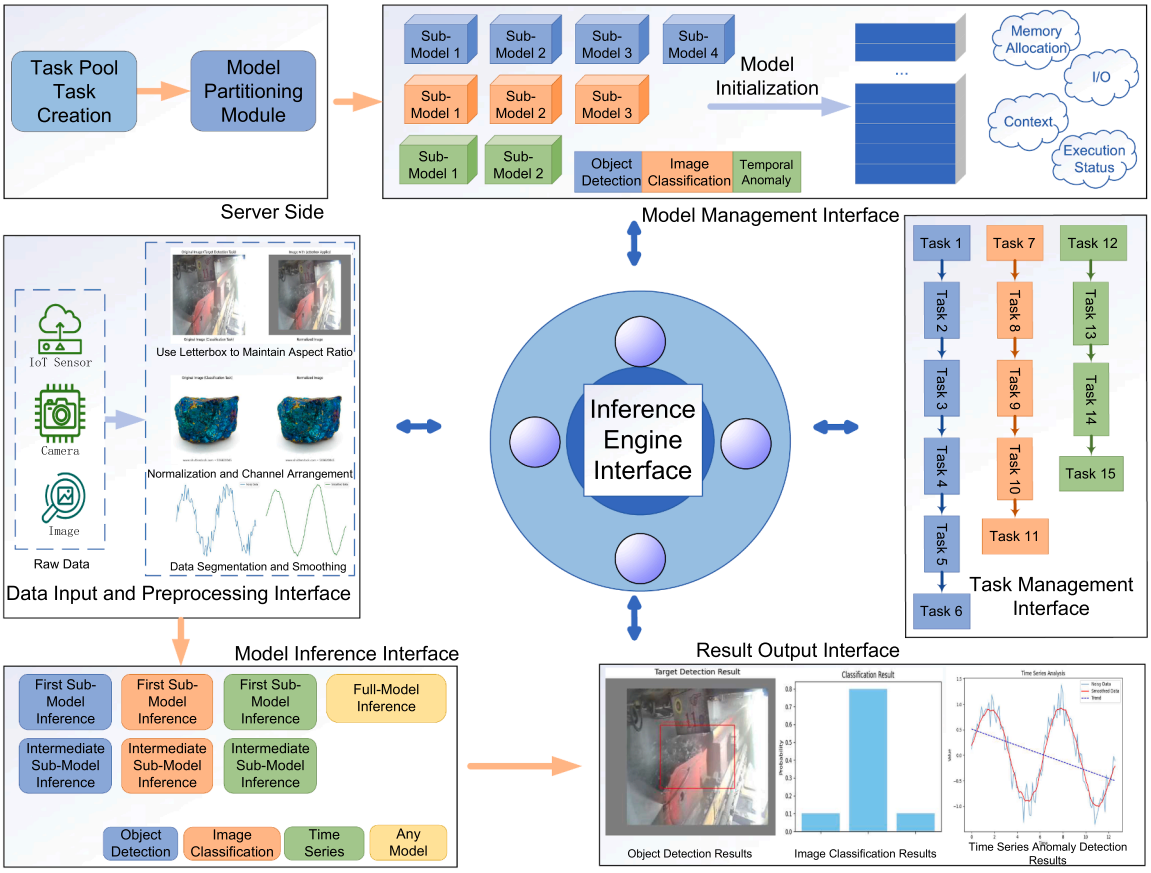


Fig. 8. Construction of inference templates.

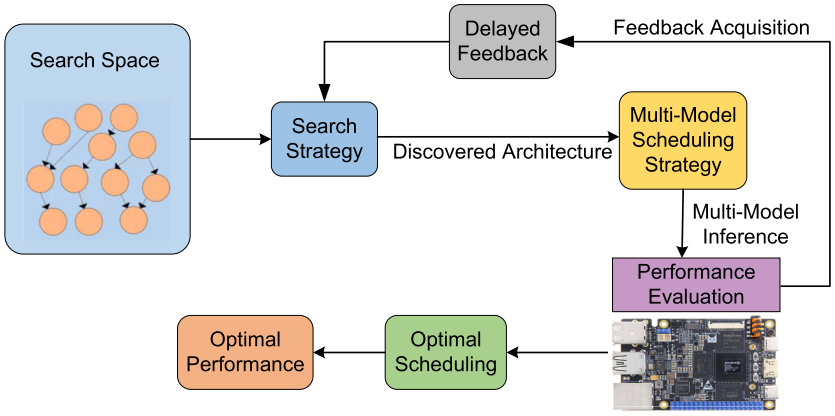


Fig. 9. Process for automatic search of optimal scheduling schemes.

Here, $\mathcal{E}^*$ represents the optimal scheduling strategy; $\mathcal{L}(\mathcal{E})$ denotes the cost model used to evaluate the inference latency under the current scheduling strategy $\mathcal{E}$, and $\mathcal{P}_{\mathcal{E}}$ defines the search space of scheduling parameters. The automatic search process for identifying the optimal scheduling strategy is illustrated in Fig. 9.

4.1. Search space

The search space is defined as the set of scheduling strategies (Salmani Pour Avval et al., 2025), comprehensively integrating the scheduling configurations of individual tasks to achieve global optimization in multi-model collaborative inference scenarios. Within the MineInfer scheduling framework, the scheduling strategy for multi-model inference tasks is determined by the collaborative integration of stage parameters (Stage), priority parameters (Priority), and NPU core parameters (NPU). The Cartesian product of these parameters constitutes a system-level search space. Each parameter not only independently controls specific scheduling attributes but also becomes interdependent through cross-model constraints, ultimately forming an integrated and system-level scheduling framework, as illustrated in Fig. 10.

To clarify the mapping between task scheduling configurations and their associated parameters, this work adopts an ordered tuple representation, denoted as $(TaskID, Stage, Priority, NPU_Core)$, to describe the complete scheduling rule of an individual task.

Specifically, the first element of the tuple represents the unique identifier of the task (*TaskID*). For example, in the configuration $(1, 1, 0, 4)$,

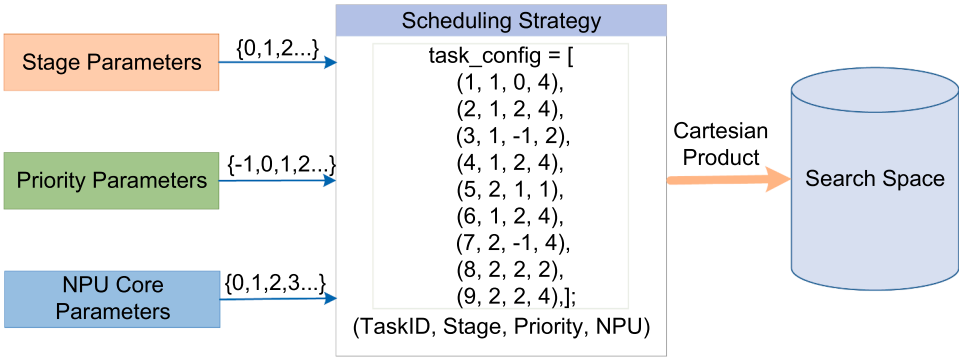


Fig. 10. Example of the search space.

the value 1 denotes the task with ID 1. The remaining elements of the tuple correspond to three scheduling parameters in order. The first is the **Execution Stage (Stage)**, which takes values from $0, 1, 2, \ldots$ and is used to partition tasks into different execution phases. The second is the **Scheduling Priority (Priority)**, which takes values from $-1, 0, 1, 2, \ldots$, where a larger value indicates a higher scheduling priority among tasks within the same execution stage. The third is the **Assigned NPU Core (NPU_Core)**, which takes values from $0, 1, 2, 3, \ldots$ and specifies the index of the NPU core to which the task is bound.

In a multi-model inference task group, each model is composed of multiple sub-models, and every sub-model is characterized by three scheduling parameters: Stage, Priority, and NPU. The search space size of each model is defined as the product of the selection space sizes of all its sub-models. Suppose the task group comprises n models, with each model consisting of m sub-models. Accordingly, the search space size of a single model is formulated in Eq. (4), while the total search space size corresponds to the product of all model-level search spaces, as specified in Eq. (5).

$$|\mathcal{P}_{\mathcal{E}}| = \left(n_{\text{stage}} \times n_{\text{priority}} \times n_{\text{NPU}}\right)^{m} \tag{4}$$

$$|\mathcal{P}_{\text{total}}| = \prod_{i=1}^{n} |\mathcal{P}_{\mathcal{E},i}| \tag{5}$$

Here, n_{stage}, n_{priority}, and n_{NPU} represent the number of possible values for the parameters stage, priority, and NPU, respectively. These parameters directly influence both the size and the complexity of the search space. Therefore, careful selection of these parameters is required to ensure the efficiency of the optimization process.

During the construction of the search space, a series of constraints must be imposed to guarantee that the generated scheduling strategies are both feasible and practical for real-world applications, including:

Concurrent stage constraint: For multiple sub-models within the same model, their stage parameters must follow a strict sequential order. Typically, the execution stage of a preceding sub-model should precede or coincide with that of a subsequent sub-model, ensuring the correctness of data dependencies and information flow. For instance, if sub-model A is a preceding sub-model of sub-model B, then the following constraint must hold:

$$A_{\text{stage}} \leq B_{\text{stage}} \tag{6}$$

Priority constraint: Within the same concurrent stage, the execution order of tasks is governed by their assigned priority. To ensure execution in the intended order, the sub-models within the same stage must have their priorities aligned with dependency constraints. For example, if Task B depends on Task A, the condition below must hold:

$$A_{\text{priority}} < B_{\text{priority}} \tag{7}$$

Mainline task constraint: In each concurrent stage, at least one mainline task is required to be designated. However, when multiple sub-models within the same model share the same concurrent stage and priority, only one may be designated as the mainline task to prevent conflicts.

4.2. Cost model

After constructing the search space, a cost model $\mathcal{L}(\mathcal{E})$ must be established to evaluate the performance of each candidate scheduling scheme. Two common approaches are employed: model-based and measurement-based methods. The model-based approach involves constructing mathematical or deep learning models to simulate task execution performance, thereby predicting execution outcomes from a given scheduling scheme. Its primary advantage is computational efficiency, making it suitable for large-scale search spaces. However, in complex computing scenarios, relying purely on model estimation may fail to fully capture actual system performance, thus limiting accuracy. In contrast, the measurement-based method evaluates scheduling schemes on real hardware, deriving accurate execution times from actual runs. This approach is particularly suitable for scenarios requiring high-precision evaluation but may incur significant computational and time overhead in large-scale search spaces. In this article, due to the complexity of resource allocation and scheduling strategies in multithreaded concurrent scheduling, accurate simulation of inference latency using a model-based method is impractical. Therefore, a measurement-based cost model construction approach is adopted, where each candidate scheduling scheme is deployed on the LubanCat-4 development board, and the average inference latency measured over multiple executions is used as the cost metric.

4.3. Scheduling algorithm and process

Given the complexity and discreteness of the search space, two search strategies—Random Search (RS) and Genetic Algorithm (GA)—are adopted to explore scheduling schemes. Both RS and GA exhibit strong global search capabilities and effectively address discrete and nonlinear scheduling problems while requiring relatively low computational resources, making them suitable for execution on resource-constrained edge devices. In contrast, gradient descent methods perform poorly on discrete and nonlinear scheduling problems and demand substantial computational resources. RS evaluates scheduling parameters through random selection, covering a broad search space and making it suitable for preliminary exploration. GA optimizes scheduling schemes via selection, crossover, and mutation, thereby achieving improved local optimization. Both methods offer distinct advantages: RS enables rapid exploration, whereas GA facilitates the discovery of superior solutions.

Initially, a random seed is set to initialize the parameters. Subsequently, the selected search algorithm—RS or GA—generates candidate scheduling schemes from the search space. For each candidate, constraint checks are performed and infeasible schemes are discarded. Thereafter, feasible candidates are evaluated by the cost model, and candidate solutions are iteratively refined based on the evaluation feedback.

If GA is employed, candidates are selected for the next generation based on their fitness; in contrast, RS retains the best-performing candidate. This procedure is iterated to progressively refine the candidates until the preset convergence criterion is met or the maximum number of iterations is reached. The overall workflow is illustrated in Algorithm 2. In this article, the search algorithm is deployed directly on the LubanCat-4 embedded development board, establishing a closed-loop workflow of "parameter generation → on-device evaluation → iterative optimization of the search," which reduces communication overhead and improves the efficiency of the scheduling-scheme search.

Algorithm 2 Scheduling scheme search process.

Require: Search space S constructed from the candidate task set $\mathcal{T}_{cand}$ and the parameter space $\mathcal{P}_e$, cost function C, algorithm type algo, max iterations max_iter, population size pop_size
Ensure: Optimal scheduling scheme schedule_opt

```
Initialize random seed
Initialize schedule_opt = (None, ∞)
Initialize candidate set P = ∅
for i = 1 to max_iter do
    if algo = Genetic Algorithm then
        if i = 1 then
            P = Randomly sample pop_size schedules from S
        else
            P = Generate new schedules via selection,
                crossover, and mutation from current P
    else if algo = Random Search then
        P = Randomly sample one schedule from S
    for each schedule s in P do
        if s satisfies constraints then
            cost = C(s)
            if cost < schedule_opt.cost then
                schedule_opt = (s, cost)
    if algo = Genetic Algorithm then
        Update P by selecting top pop_size schedules based on cost
Return schedule_opt
```

The space complexity of the scheduling search is primarily influenced by the total search space $|\mathcal{P}_{total}|$, which, as indicated in Eq. (5), grows exponentially with the number of sub-models m; however, since our practical implementation only maintains the current optimal solution and the candidate population, the actual space complexity remains $O(P \times K)$, where P denotes the population size and K represents the total number of scheduling tasks. Furthermore, the time complexity predominantly depends on the selected search strategy, where the complexity for Random Search (RS) is $O(I \times C)$ and for the Genetic Algorithm (GA) is $O(I \times P \times C)$, with I being the maximum number of iterations and C representing the time overhead of a single cost function evaluation through on-device latency measurement.

5. Experimental design and analysis

This article adopts two search strategies-Random Search (RS) and Genetic Algorithm (GA)-to explore scheduling schemes. A series of monitoring models are composed into distinct multi-model inference task groups. For each combination, the optimal scheduling scheme is identified. The identified optimal scheduling schemes are then compared with three multi-model inference approaches: serial inference, simple multi-concurrency, and modular concurrency.

5.1. Experimental setup

The experimental platform in this section is based on the LubanCat-4 development board, equipped with three independent NPU cores. The detailed configuration of the LubanCat-4 is summarized in Table 1.

Table 1
Detailed configuration of LubanCat-4.

Configuration	LubanCat-4
SoC	RockChip RK3588s
CPU	4 x ARM Cortex-A76 + 4 x ARM Cortex-A55
NPU	3 x 2 TOPS (INT8) inference performance
Video Processing	8K @ 60 fps decoding / 8K @ 30 fps encoding
Memory	LPDDR4X, 4 GB
Storage	32 GB eMMC
Peripheral Interfaces	Gigabit Ethernet, mini-HDMI, USB 3.0, MIPI

Although multiple NPU cores provide additional computational resources, if the sub-models of a single task are concurrently distributed across multiple cores, inter-core communication becomes inevitable, potentially introducing performance overhead. For relatively simple models, a single-core execution may be more efficient. Therefore, the experiments in this section are conducted under two NPU utilization modes: (1) single-core execution and (2) multi-core execution. This article employs three multi-model scheduling strategies as baseline methods for the experiments:

- **Serial Inference**: The default strategy provided by the official Rockchip RKNN (Airockchip, 2025) deployment framework. In this strategy, models are executed sequentially without parallelism, serving as the reference baseline.
- **Multithreaded Concurrent Inference**: The framework is implemented using the lightweight, multithreaded, high-concurrency architecture of RKNN. Each model is assigned an independent thread, enabling multiple threads to execute in parallel. This design is analogous to the multi-stream concurrency mechanism commonly adopted in GPUs (Huang et al., 2020).
- **Modular Concurrent Inference**: Implemented on a coarse-grained concurrent inference framework (Nndeploy, 2025). This framework partitions inference tasks into three stages—preprocessing, inference, and postprocessing—which are scheduled concurrently using a thread pool.

5.2. Multi-model inference task pool

The task pool constructed in this article comprises six representative mine monitoring tasks: anomaly detection of hydraulic protective plates in fully mechanized mining faces (MineDetV11), coal miner behavior recognition (MineDetV11), equipment monitoring on drilling platforms (YoloV11m), foreign object classification on coal conveyor belts (MineClass-n), ore classification (MineClass-n), and gas concentration early warning (MineLSTM).

Among these, MineDetV11, MineClass-n, and MineLSTM are derivative models obtained through neural architecture search (NAS) based on widely used public baseline models, namely YOLOv11n (Jocher et al., 2024), ResNet50 (He et al., 2016), and CNN-LSTM, respectively, with additional RepConv-based structural optimization. These models were developed in our laboratory under a self-designed MineNAS framework and are tailored to distinct perception tasks, including object detection (MineDetV11), image classification (MineClass-n), and time series forecasting (MineLSTM). During model design, the overall network topology and operator types of the corresponding baseline models are largely preserved, while NAS is employed to explore lightweight architectural variants and RepConv modules are introduced to further enhance inference efficiency and suitability for edge deployment.

Table 2 shows the comparative experimental results for coal miner behavior recognition. The MineDetV11 model achieves the best trade-off between detection accuracy and inference latency.

The image classification results are summarized in Table 3. MineClass-n achieves the best balance between classification accuracy and inference speed in both the conveyor-belt foreign object detection task and the ore classification task, where Task A denotes the

Table 2
Performance comparison of object detection models.

Model	mAP@50 (%)	mAP@50-95 (%)	Params /M	FLOPs /G	Inference Latency/ms
YOLOv11	91.2	**73.9**	2.6	6.3	**30.5**
MineDetv11	92.0	**74.4**	3.5	10.5	**26.5**

Table 3
Performance comparison of image classification models.

Model	Accuracy (%)		Params /M		FLOPS /G		Inference Latency/ms	
	A	B	A	B	A	B	A	B
RESNET50	88.9	90.1	26.1	26.1	68.8	68.8	5.4	5.4
MineClass-n	**91.6**	**91.8**	3.6	3.5	7.9	6.2	**1.7**	**1.69**

Table 4
Performance comparison of time-series prediction models.

Model	MAE	RMSE	R^2	Params	Inference (Latency/ms)
CNN_LSTM	0.0164	0.0252	0.9239	103,427	5.7
MineLSTM	0.0151	0.0233	0.9354	131,267	5.9

conveyor-belt foreign object classification task and Task B denotes the ore classification task.

The experimental results for the time-series anomaly early-warning task are shown in Table 4. MineLSTM achieves similar predictive performance and inference latency to CNN_LSTM.

Overall, the experimental results indicate that, under identical hardware platforms and inference frameworks, the Mine-series models consistently outperform their corresponding baseline models in terms of inference latency and overall execution efficiency. Given the high degree of architectural consistency between the Mine-series models and their baselines, these performance improvements mainly stem from structural optimization and lightweight design, rather than differences in scheduling strategies or system configurations. From the perspective of scheduling and system execution, the Mine-series models exhibit execution behaviors on the MineInfer framework that are consistent with those of the corresponding mainstream baseline models, with unchanged scheduling characteristics and resource contention patterns.

To comprehensively evaluate the scheduling performance of the proposed MineInfer framework under diverse task compositions and load conditions, seven task scenarios were constructed from the task pool.

- **Low-Load Workload Set** (MineClass-n × 2 + MineLSTM × 1): evaluates inference latency under minimal computational demands.
- **Medium-Load Workload Set** (MineDetV11 × 2 + MineClass-s × 1): assesses scheduling efficiency under moderately unbalanced task complexity.
- **High-Load Workload Set** (MineDetV11 × 2 + YoloV11m × 1): examines system responsiveness under intensive computational pressure.
- To reflect practical scenarios involving heterogeneous task scheduling, two **Multi-Type Workload Sets** were constructed:
  - **Multi-Type Workload Set A** (2 × MineDetV11 + 1 × MineClass-s + 1 × MineLSTM): composed of mixed tasks, used to evaluate scheduling behavior under medium-to-light task heterogeneity.
  - **Multi-Type Workload Set B** (2 × MineDetV11 + 1 × MineClass-s + 1 × YoloV11m): composed of mixed tasks, used to examine scheduling performance under medium-to-heavy task mixtures.
- **Mixed-Load Workload Set** (MineDetV11 × 2 + MineClass-s × 2 + YoloV11m × 1): assesses robustness under concurrent high load and task diversity.
- **Full-Load Workload Set** (covering all six task types): validates MineInfer under extreme multi-task concurrency.

When identical model names appear within a task set, they correspond to distinct task instances rather than duplicated execution threads.

5.3. Inference latency and inference speedup ratio

5.3.1. Single-core NPU execution

To clearly demonstrate the acceleration effect of the proposed framework on inference latency, the serial inference framework provided by RKNN was employed as the experimental baseline. The inference speedup ratio was computed by normalizing the measured inference latency. As illustrated in Fig. 11, the proposed inference framework achieved the lowest inference latency across all seven task combinations in the single-core experiments. The achieved inference speedup ranged from 1.3-2.53×, consistently surpassing the performance of all baseline methods.

Further analysis indicates that, under the Low-Load Configuration, all inference strategies achieved the highest inference speedup. This can be attributed to the fact that, under this configuration, computational resources are not saturated, enabling the system to attain higher inference speedups through increased concurrency. For other workload configurations, the inference speedups of multithreaded and modular concurrent strategies remained relatively low, ranging from 1.1-1.4×. This limitation is due to the fact that both strategies submit all operations for direct execution without fine-grained interleaved scheduling, resulting in ineffective utilization of system resources.

In contrast, the proposed MineInfer framework incorporates stage-level concurrency and intra-stage priority scheduling within the computation graph scheduling process. This design enables interleaved execution, thereby maximizing computational resource utilization. Furthermore, scheduling schemes derived from the Genetic Algorithm (GA) demonstrated higher inference efficiency than those obtained through Random Search (RS). This is due to GA refining the solution space through its convergence process, while RS selects solutions randomly and entirely lacks any optimization procedure. The detailed inference latency results for each task combination are presented in Table 5.

5.3.2. Multi-core NPU execution

As shown in Fig. 12, during the multi-core NPU experiments, the MineInfer framework consistently outperformed other inference methods in terms of inference speedup ratio when evaluated under the optimal scheduling schemes derived from RS and GA.

Compared with the single-core NPU environment, the multi-core architecture achieves a higher speedup ratio. This improvement can be attributed to two main factors: First, increasing the number of NPU cores expands the dimensionality of hardware parameter configurations, resulting in a geometrically larger search space that covers more potential high-performance architectural combinations. Second, the additional NPU cores enhance the overall computing capacity, allowing idle resources to be reallocated to Low-Load configurations during the execution of complex workloads. However, compared with the single-core scenario, the convergence of the scheduling search process in the multi-core architecture is slower, thereby requiring a longer search time. This slowdown arises from the increased complexity of the multi-core search space, which involves more parameters and architectural combinations, thereby prolonging the time required for the algorithm to converge to the global optimum. Table 6 presents the inference latency results for each workload configuration in the multi-core NPU experiments.

5.4. Determination of hyperparameter ranges

During the scheduling scheme optimization process, the optimal Stage, Priority, and NPU parameters for each inference task are automatically searched. However, the corresponding parameter ranges-n_{stage},

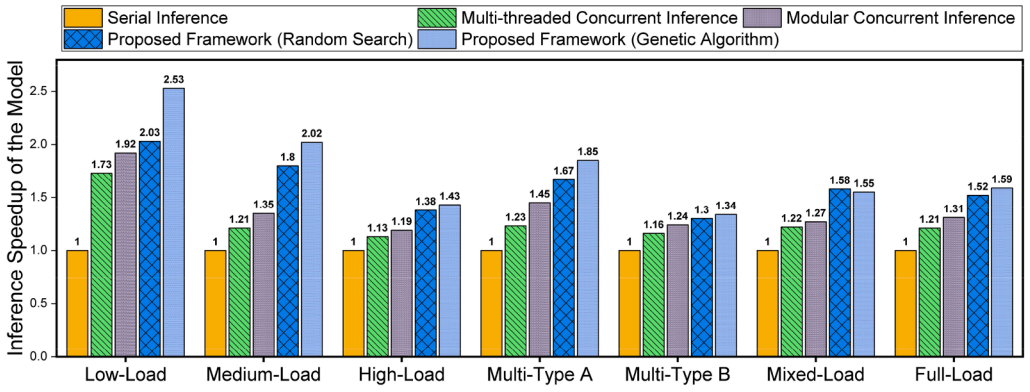


Fig. 11. Inference speedup under single-core execution.

Table 5
Inference latency in single-core scenarios.

Workload Sets	RKNN Serial Inference/ms	Multi-threaded Concurrent Inference / ms	Modular Concurrent Inference/ms	Proposed Framework (RS)/ms	Proposed Framework (GA)/ms
Low-Load	12.83	7.395	6.691	6.316	5.068
Medium-Load	53.37	44.251	39.5684	29.635	26.396
High-load	147.35	130.331	123.883	106.929	102.71
Multi-Type A	58.24	47.251	40.1253	34.794	31.564
Multi-Type B	134.45	115.572	108.094	103.548	100.365
Mixed-Load	158.67	130.505	124.859	100.479	102.129
Full-Load	164.53	136.107	125.378	108.258	103.794

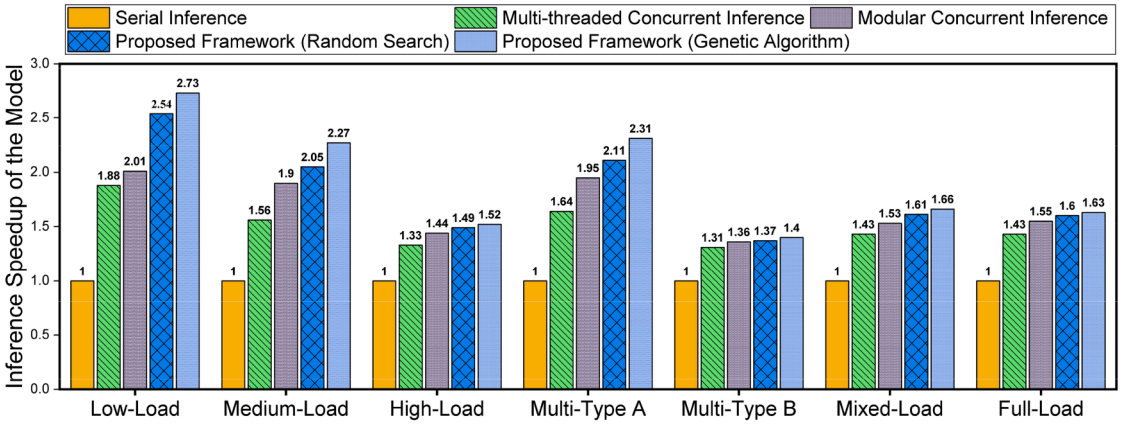


Fig. 12. Inference speedup under multi-core execution.

n_{priority}, and n_{NPU}-must still be manually specified. Among these, the value of n_{NPU} is constrained by the number of NPU cores available on the development board. During the scheduling parameter search, the optimization focuses on efficient allocation of NPU resources among tasks, rather than adjusting the number of NPU cores themselves. Therefore, in this section, the value range of n_{NPU} is not analyzed.

Under the single-core operating scenario, two types of workload configurations are examined as representative cases: the Medium-Load Configuration and the Full-Load Configuration. The optimal values of n_{stage} and n_{priority} are identified using both Random Search (RS) and the Genetic Algorithm (GA). The detailed experimental setup is presented in Table 7. For each experimental configuration, the scheduling search is performed for 200 iterations, during which the scheduling solution obtained at each iteration and the corresponding inference latency are recorded in the configuration phase. After completion of the search, the optimal scheduling configuration is applied to subsequent inference executions under the same task combination.

The experimental results of Random Search (RS) applied to the Medium-Load Configuration are presented in Fig. 13. In Fig. 13(a), a scatter plot of search rounds versus inference latency is shown, where the x-axis denotes the number of search rounds and the y-axis denotes the inference latency. From the figure, it can be observed that during the RS process, the inference latency of the scheduling schemes does not decrease or exhibit any convergence trend with increasing search rounds. Fig. 13(b) illustrates the probability density distribution of inference latency, with a fitted normal distribution applied to the data. The results indicate that the RandSP-12 scheme achieves the best performance, yielding a mean inference latency of 41.493 ms and a standard deviation of 7.673 ms, showing lower mean latency and higher stability compared with the other schemes.

The experimental results of the Genetic Algorithm (GA) search for the Medium-Load Configuration are presented in Fig. 14. As shown in Fig. 14(a), the inference latency progressively decreases with the number of search rounds, ultimately reaching a minimum of 26.396 ms

Table 6
Inference latency in multi-core scenarios.

Workload Sets	RKNN Serial Inference/ms	Multi-threaded Concurrent Inference / ms	Modular Concurrent Inference/ms	Proposed Framework (RS)/ms	Proposed Framework (GA)/ms
Light-Load	12.66	6.731	6.2919	4.977	4.631
Medium-Load	52.94	34.026	27.9117	25.875	23.313
High-Load	142.06	106.637	98.8889	95.626	93.711
Multi-Type A	57.57	35.043	29.467	27.236	24.894
Multi-Type B	131.47	100.227	96.6637	96.011	94.241
Mixed-Load	156.73	109.9	102.145	97.259	94.412
Full-Load	161.30	112.412	104.186	100.602	99.022

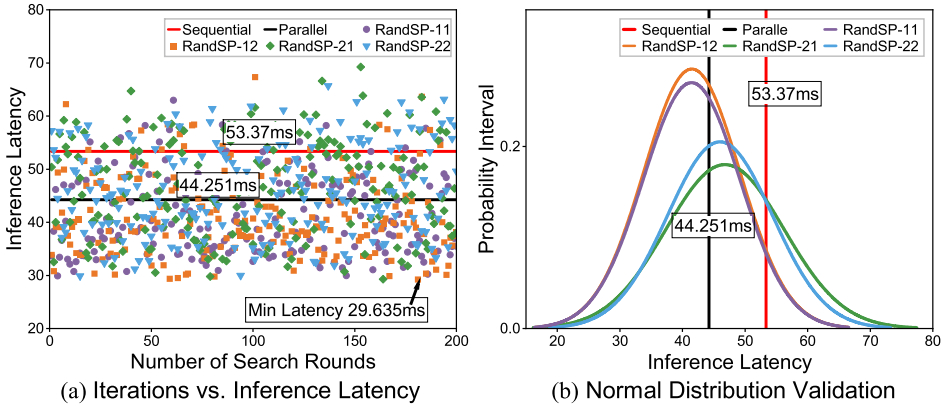


Fig. 13. Random search performance under the medium-load configuration.

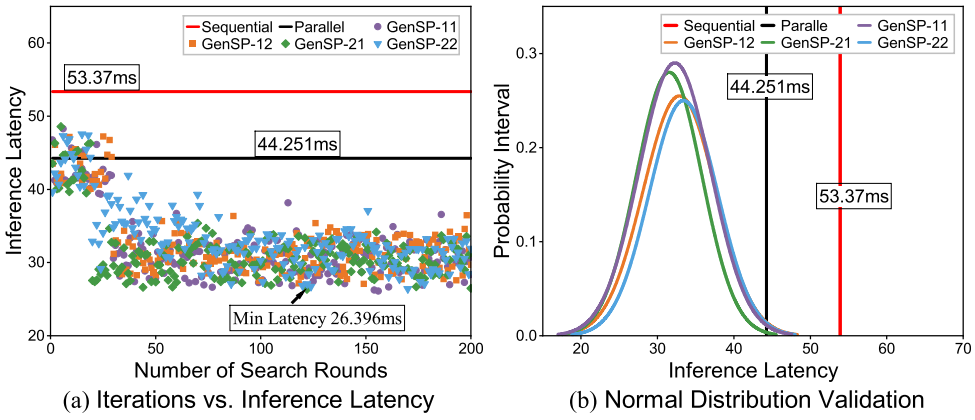


Fig. 14. Genetic algorithm performance under the medium-load configuration.

Table 7
Hyperparameter determination experimental setup.

Scheme	Search Method	Stage Value Range	Priority Value Range
RandSP-11	Random Search	{0, 1}	{–1, 0, 1}
RandSP-12	Random Search	{0, 1, 2}	{–1, 0, 1, 2}
RandSP-21	Random Search	{0, 1, 2}	{–1, 0, 1}
RandSP-22	Random Search	{0, 1, 2}	{–1, 0, 1, 2}
GenSP-11	Genetic Algorithm	{0, 1}	{–1, 0, 1}
GenSP-12	Genetic Algorithm	{0, 1, 2}	{–1, 0, 1, 2}
GenSP-21	Genetic Algorithm	{0, 1, 2}	{–1, 0, 1}
GenSP-22	Genetic Algorithm	{0, 1, 2}	{–1, 0, 1, 2}

during the GA search. Fig. 14 (b) illustrates the probability density distribution of inference latency, where the GenSP-21 scheme achieves the best performance. This scheme achieves a mean inference latency of 31.53 ms and a standard deviation of 4.30, indicating superior performance and stability.

To comprehensively evaluate the effectiveness of the scheduling scheme optimization, further experiments were conducted on the Full-Load Configuration using both search algorithms. The experimental results obtained with the Random Search (RS) and Genetic Algorithm (GA) are presented in Figs. 15 and 16, respectively. During the GA-based search process, the scheduling strategy exhibits a clear optimization trend as the number of search rounds increases, resulting in a gradual reduction in overall inference latency. Although complete convergence is not achieved, the minimum latency reaches 103.79 ms, demonstrating a distinct advantage over the 108.26 ms achieved by RS. The probability density distribution of inference latency under RS indicates that the RandSP-11 scheme achieves the best performance, with a mean latency of 129.03 ms and a standard deviation of 9.67. In the GA experiment, the GenSP-21 scheme achieves the best performance, with a mean latency

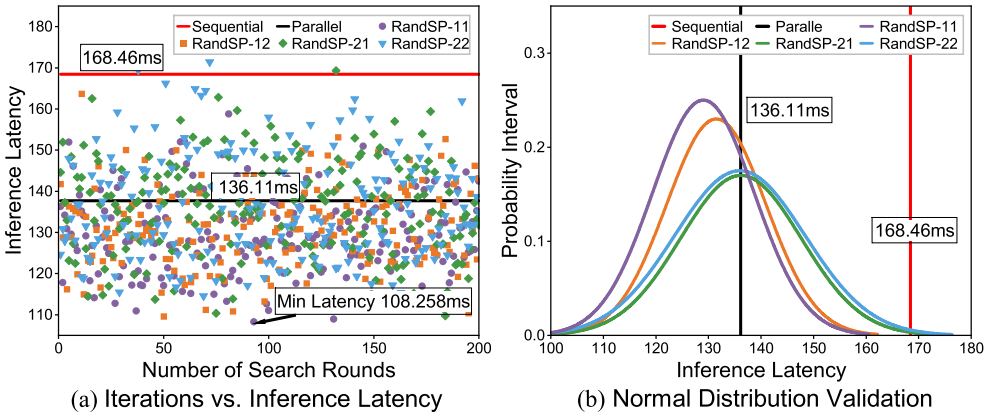


Fig. 15. Random search performance under the full-load configuration.

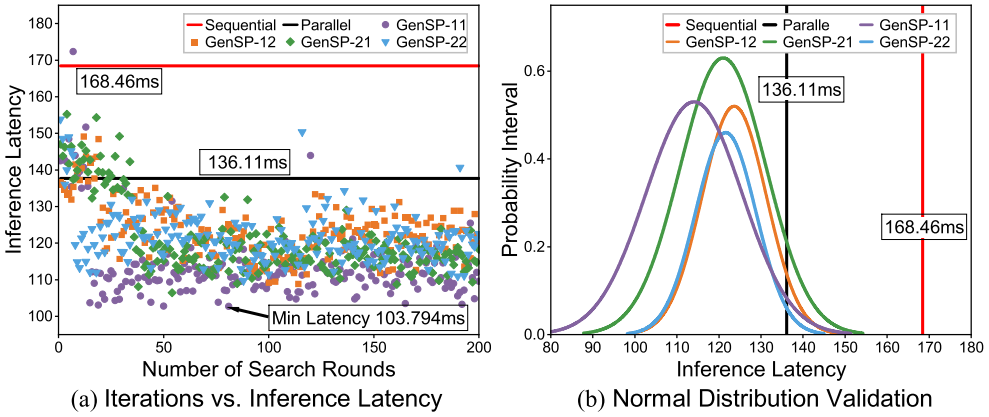


Fig. 16. Genetic algorithm performance under the full-load configuration.

of 120.99 ms and a standard deviation of 10.21. This finding further confirms that GA optimizes scheduling strategies more effectively, particularly in resource-constrained environments with stringent latency requirements.

To further compare the performance of Random Search (RS) and the Genetic Algorithm (GA) in scheduling strategy optimization, 200 search rounds of data were collected within the optimal hyperparameter ranges for the Medium-Load and Full-Load Configuration, respectively. The inference latency results were subsequently ranked in descending order. Subsequently, a surface fitting analysis was performed to evaluate the search efficiency and performance differences of the two strategies within the optimal parameter ranges. Fig. 17(a) and (b) illustrate the comparison results for the Medium-Load and Full-Load Configuration, respectively. From these results, it can be observed that under both configurations, the GA achieves faster convergence than RS and yields superior scheduling solutions. This occurs because the GA employs selection, crossover, and mutation operations to preserve high-quality individuals while maintaining population diversity. This mechanism effectively avoids local optima and accelerates convergence toward a globally optimal solution. In contrast, RS relies solely on random sampling without heuristic guidance, which renders its exploration of the search space unguided and inefficient, leading to slower convergence and reduced optimization performance.

5.5. Cross-platform applicability

Although the proposed MineInfer framework is currently instantiated and validated on the Rockchip RK3588 NPU platform, its underlying scheduling methodology is not inherently restricted to this specific hardware backend. The main objective of MineInfer is to provide a unified scheduling framework for collaborative multi-model inference under resource-constrained edge environments, rather than to optimize only a platform-specific RKNN execution interface.

In particular, the “NPU core” parameter used in this work should be interpreted more generally as a heterogeneous computing resource dimension. On the RK3588 platform, it corresponds to NPU cores; on other edge platforms, however, it can be naturally mapped to GPU execution resources, CPU cores or threads, or other accelerator resource slots, depending on the available runtime environment. From this perspective, the proposed scheduling formulation is defined at the task and resource allocation level, rather than at the level of platform-specific operators.

Accordingly, several key components of MineInfer are transferable across heterogeneous edge platforms. First, the framework adopts a fine-grained task abstraction by partitioning original models into schedulable sub-model or sub-task units, which provides the granularity required for concurrent execution and resource optimization. Second, execution constraints are modeled through dependency graphs, which ensures the correctness of task ordering and data dependencies during scheduling. Third, MineInfer defines a unified scheduling search space based on stage partitioning, priority assignment, and resource allocation, offering a common representation for combinatorial scheduling optimization. Fourth, the framework relies on profiling-based cost modeling and search, enabling it to identify effective scheduling configurations without requiring a fully hand-crafted analytical model.

In practice, the parts that require platform-specific adaptation are mainly the backend inference runtime interface, the resource accounting mechanism, and the hardware-dependent terms in the profiling-based cost model. By contrast, the scheduling abstraction itself, including task

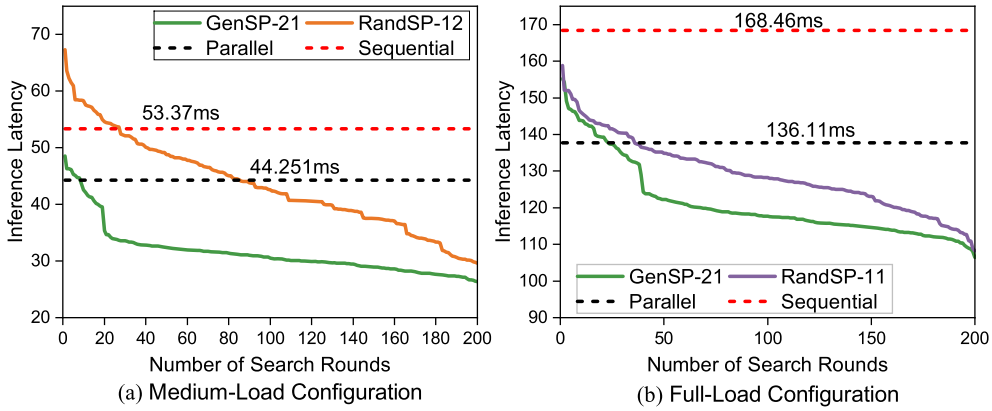


Fig. 17. Comparison of search strategies under different load configurations.

decomposition, dependency modeling, and search-space construction, remains unchanged across platforms.

6. Conclusion

This article has presented MineInfer, a lightweight framework for automatic model design and multi-model inference scheduling, specifically tailored for edge devices with constrained computational resources. To maximize hardware utilization, MineInfer integrates a scheduling-parameter-search-based scheduling optimization strategy that constructs a three-dimensional search space defined by execution stage, task priority, and NPU allocation. By leveraging Random Search and Genetic Algorithms, the framework enables online adaptive scheduling, effectively enhancing deployment robustness under dynamic workloads. Through fine-grained concurrency, load balancing, and an abstract interface design, MineInfer achieves efficient collaborative inference while minimizing deployment and maintenance overhead. Extensive experimental evaluations demonstrate that MineInfer significantly reduces inference latency and improves resource utilization, consistently outperforming conventional inference schemes on representative edge platforms. These results confirm the effectiveness and generality of the proposed approach, indicating its strong potential for diverse edge computing scenarios and deployment requirements. Beyond the RK3588-based deployment, MineInfer provides a reusable system-level methodology-covering execution organization, runtime support, and profiling-guided scheduling optimization-that can be reconstructed for other multi-model edge workloads and NPU platforms with lightweight adaptation.

Although this article has proposed MineInfer as a system-level framework for multi-model collaborative inference and scheduling optimization on resource-constrained edge devices, several directions remain for future work. First, we will enhance the scheduling-parameter-search strategy by integrating Bayesian optimization and online-feedback-guided adaptive search to accelerate convergence and reduce search overhead. Second, future work will further validate and extend MineInfer across additional edge NPU platforms and other heterogeneous hardware architectures, including GPU-based edge modules and multi-core CPU-only systems, under more complex and dynamic workload scenarios, in order to further assess its adaptability, robustness, and generalizability.

CRediT authorship contribution statement

Lei He: Conceptualization, Formal analysis, Validation, Data curation, Methodology, Writing – original draft, Writing – review & editing. Lei He took the lead in conceptualizing and designing the study, developing the methodology, writing the original draft, and performing the majority of the data collection, formal analysis, validation, and experiment design. She also led the results analysis and literature review; **Yuelong Li:** Software, Investigation, Data curation, Investigation, Validation, Writing – original draft. Yuelong Li was responsible for the development of the software system, conducted field investigations and data curation work, participated in the validation of experimental results, and contributed to the writing of the original draft; **Weijun Cheng:** Supervision, Project Administration, Funding Acquisition, Writing – Review & Editing, Methodology, Conceptualization. Weijun Cheng supervised the research project, provided administrative support, secured funding, and contributed to the methodology and conceptualization. He also reviewed and edited the manuscript extensively.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Weijun Cheng reports financial support was provided by National Natural Science Foundation of China (52374169) and the Fundamental Research Funds for the Central Universities. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Airockchip (2025). rknn-toolkit2. [EB/OL]. Accessed: January 10, 2025. https://github.com/airockchip/rknn-toolkit2/tree/master/doc.

Ali, M., & Pal, I. (2022). Assessment of workers' safety behavior in the extractive industries: The case of underground coal mining in Pakistan. *The Extractive Industries and Society*, *10*(1), 101087.

Chang, H., Meng, X., Wang, X., & Hu, Z. (2024). Research on coal mine longwall face gas state analysis and safety warning strategy based on multi-sensor forecasting models. *Scientific Reports*, *14*(1), 13795.

Cheng, J., Jiang, J., Kang, H., & Ma, L. (2025). A hybrid neural architecture search algorithm optimized via lifespan particle swarm optimization for coal mine image recognition. *Mathematics*, *13*(4), 631.

Duan, C., & Xu, Y. (2024). Insulation life prediction of stacking multi-model fusion motors based on Conv1d-LSTM. In *Journal of Physics: Conference Series*, *2797*, 012014, IOP Publishing .

Fan, Y., Mao, S., Li, M., Wu, Z., & Kang, J. (2024). CM-YOLOv8: Lightweight YOLO for coal mine fully mechanized mining face. *Sensors*, *24*(6), 1866.

Hanif, M. W., Li, Z., Yu, Z., & Bashir, R. (2024). A lightweight object detection approach based on edge computing for mining industry. *IET Image Processing*, *18*(13), 4005–4022.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR 2016)* (pp. 770–778). Las Vegas, NV, USA.

Huang, Y., Zhang, Y., Feng, B., Guo, X., Zhang, Y., & Ding, Y. (September 2020). A close look at multi-tenant parallel CNN inference for autonomous driving. In *IFIP International conference on network and parallel computing* (pp. 92–104). Cham: Springer International Publishing.

Jeong, J. S., Lee, J., Kim, D., Jeon, C., Jeong, C., Lee, Y., & Chun, B. G. (June 2022). Band: Coordinated multi-dnn inference on heterogeneous mobile processors. In *Proceedings of the 20th annual international conference on mobile systems, applications and services* (pp. 235–247).

Jocher, G., Qiu, J., & Chaurasia, A. (2024). Yolov11. [EB/OL]. Accessed: May 11, 2024. https://github.com/ultralytics/ultralytics/tree/main.

Kim, S., Genc, H., Nikiforov, V. V., Asanović, K., Nikolić, B., & Shao, Y. S. (February 2023). Moca: Memory-centric, adaptive execution for multi-tenant deep neural networks. In *2023 IEEE international symposium on high-performance computer architecture (HPCA)* (pp. 828–841). IEEE.

Kou, Q., Ma, H., Xu, J., Jiang, H., & Cheng, D. (2023). Coal flow foreign body classification based on ESCBAM and multi-channel feature fusion. *Sensors*, *23*(15), 6831.

Lalitha, K., Ramya, G., & Shunmugathammal, M. (2023). AI-based safety helmet for mining workers using IoT technology and ARM cortex-M. *IEEE Sensors Journal*, *23*(18), 21355–21362.

Li, Y., Wang, W., Cheng, W., & Nie, G. (August 2024). MineDet: A real-time object detection framework based neural architecture search for coal mines. In *International conference on intelligent computing* (pp. 30–41). Singapore: Springer Nature Singapore.

Liu, Y., Meng, X., Zhou, K., & Zhang, Y. (2025). Internet of things-enabled active safe mining: Architecture and case studies. *Measurement*, (p. 118987).

Liu, Z., Li, L., Zeng, J., Wang, Y., Yang, J., & Liu, X. (2024). Predicting the ash content in coal flotation concentrate based on convolutional neural network. *International Journal of Coal Preparation and Utilization*, *44*(12), 2080–2096.

Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., & Guo, B. (2021). Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 10012–10022).

Luan, H., Xu, H., Tang, W., Tian, Y., & Zhang, Q. (2023). Coal and gangue classification in actual environment of mines based on deep learning. *Measurement*, *211*, 112651.

Nndeploy (2025). nndeploy. [EB/OL]. Accessed January 6, 2025. https://github.com/nndeploy/nndeploy.

Salmani Pour Avval, S., Eskue, N. D., Groves, R. M., & Yaghoubi, V. (2025). Systematic review on neural architecture search. *Artificial Intelligence Review*, *58*(3), 73.

Shi, W., Li, J., Chen, Z., Zhang, C., & Yang, H. (December 2024). Smart mine: The role of artificial intelligence in modern coal extraction. In *2024 IEEE/WIC international conference on web intelligence and intelligent agent technology (WI-IAT)* (pp. 874–879). IEEE.

Sun, Y., & Ji, F. (2025). An embodied intelligence system for coal mine safety assessment based on multi-level large language models. *Sensors*, *25*(2), 488.

Wang, Y., Wang, Y., & Dang, L. (2023). Video detection of foreign objects on the surface of belt conveyor underground coal mine based on improved SSD. *Journal of Ambient Intelligence and Humanized Computing*, *14*(5), 5507–5516.

Wu, J., Zheng, R., Jiang, J., Tian, Z., Chen, W., Wang, Z., & Leung, V. C. (2024). A lightweight small object detection method based on multilayer coordination federated intelligence for coal mine IOVT. *IEEE Internet of Things Journal*, *11*(11), 20072–20087.

Xu, J., Yan, C., Boota, M. W., Chen, X., Li, Z., Liu, W., & Yan, X. (2024). Research on automatic identification of coal mining subsidence area based on InSAR and time series classification. *Journal of Cleaner Production*, *470*, 143293.

Xue, Y., Liu, Y., Nai, L., & Huang, J. (June 2023). V10: Hardware-assisted NPU multi-tenancy for improved resource utilization and fairness. In *Proceedings of the 50th annual international symposium on computer architecture* (pp. 1–15).

You, M., Li, S., Li, D., & Xu, S. (2021). Applications of artificial intelligence for coal mine gas risk assessment. *Safety Science*, *143*, 105420.

Yu, F., Bray, S., Wang, D., Shangguan, L., Tang, X., Liu, C., & Chen, X. (November 2021). Automated runtime-aware scheduling for multi-tenant DNN inference on GPU. In *2021 IEEE/ACM International conference on computer aided design (ICCAD)* (pp. 1–9). IEEE.

Zhang, X., Chen, X., Ding, Y., Zhang, Y., Wang, Z., Shi, J., & Huang, X. (2024a). Smart real-time evaluation of tunnel fire risk and evacuation safety via computer vision. *Safety Science*, *177*, 106563.

Zhang, Y., Qin, Y., Zhang, Y., Zhou, X., Jian, S., Tan, Y., & Li, K. (2024b). OnceNAS: Discovering efficient on-device inference neural networks for edge devices. *Information Sciences*, *669*, 120567.

Zhang, Z., Li, H., Zhao, Y., Lin, C., & Liu, J. (May 2023). POS: An operator scheduling framework for multi-model inference on edge intelligent computing. In *Proceedings of the 22nd international conference on information processing in sensor networks* (pp. 1–1).

Zhang, Z., Zhao, Y., Li, H., & Liu, J. (2024c). BCEdge: SLO-aware DNN inference services with adaptive batch-concurrent scheduling on edge devices. *IEEE Transactions on Network and Service Management*, *21*(4), 4131–4145.

Zhao, Z., Ling, N., Guan, N., & Xing, G. (November 2023). Miriam: Exploiting elastic kernels for real-time multi-DNN inference on edge GPU. In *Proceedings of the 21st ACM conference on embedded networked sensor systems* (pp. 97–110).

Zou, A., Li, J., Gill, C. D., & Zhang, X. (2023). RTGPU: Real-time GPU scheduling of hard deadline parallel tasks with fine-grain utilization. *IEEE Transactions on Parallel and Distributed Systems*, *34*(5), 1450–1465.